final-melhorado-v2

June 18, 2026

[1]:
```
'''
Bibliotecas usadas
'''

import os
from datetime import datetime
import re
import subprocess
import warnings
import numpy as np
import pandas as pd
from collections import Counter
from Bio import Entrez
from Bio import SeqIO
from Bio.Seq import Seq
from sklearn.model_selection import (
    GroupShuffleSplit,
    StratifiedGroupKFold,
    RandomizedSearchCV,
    GroupKFold
    )
from sklearn.metrics import (
    classification_report,
    roc_auc_score,
    average_precision_score,
    make_scorer,
    matthews_corrcoef,
    f1_score,
    precision_score,
    recall_score,
    balanced_accuracy_score,
    accuracy_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_curve,
    auc
)
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_validate
from sklearn.svm import SVC
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from xgboost import XGBClassifier
import matplotlib.pyplot as plt
import subprocess
import shap
import seaborn as sns
from sklearn.inspection import permutation_importance


warnings.filterwarnings("ignore")

print("Bibliotecas importadas")
```

```
Bibliotecas importadas
```

[2]:
```
'''
obtenção dos datasets
'''

EMAIL = "marcela.leite@gmail.com.br"
OUTPUT_DIR = "pipelineAG"
os.makedirs(OUTPUT_DIR, exist_ok=True)
Entrez.email = EMAIL

# ============================================================
# QUERIES
# ============================================================

POSITIVE_QUERY = r'''
    ("Solanaceae"[Organism] NOT ("Solanum betaceum"[Organism] OR "Solanum␣
 ↪melongena"[Organism] OR "Solanum tuberosum"[Organism] OR "Physalis␣
 ↪peruviana"[Organism]))
    AND (TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA␣
 ↪OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
    OR "Rx1" OR "Sw-5" OR "Tm-2" OR "Tm-22" OR "RCY1" OR "Rx" OR "N GENE"
    OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR" OR "CC-NBS-LRR" OR "TNL"␣
 ↪OR "CNL Domain" OR "CNL"
    OR "NBS-ARC" OR "NB-ARC domain" OR "NB-ARC" OR "Nucleotide-binding site"
    OR "RDR" OR "Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" )
    NOT (partial OR fragment OR hypothetical OR predicted OR DAP-seq OR␣
 ↪"transcription factor" OR DNA-binding OR microtom)
```

```
    NOT ("pathogenesis-reletad protein" OR osmotin OR PR1 OR PR2 OR PR3 OR PR4␣
 ↪OR PR5 OR synthase OR metabolic OR precursor)
    NOT txid10239[Organism:exp]
'''
NEGATIVE_QUERY = r'''
    ("Solanaceae"[Organism] NOT ("Solanum melongena"[Organism] OR "Solanum␣
 ↪tuberosum"[Organism] OR "Physalis peruviana"[Organism]))
   AND 100:3000[Sequence Length]
   NOT (TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA␣
 ↪OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
    OR "Rx1" OR "Sw-5" OR "Tm-2" OR "Tm-22" OR "RCY1" OR "Rx" OR "N"
    OR "virus resistance" OR "antiviral" OR "Tobacco mosaic virus" OR "Potato␣
 ↪virus"
    OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR" OR "CC-NBS-LRR" OR "TNL"␣
 ↪OR "CNL Domain" OR "CNL"
    OR "NBS-ARC" OR "NB-ARC domain" OR "NB-ARC" OR "Nucleotide-binding site"
    OR "RNA silencing" OR "RDR" OR "Dicer-like" OR "Argonaute" OR "AGO" OR␣
 ↪"eIF4E" )
   NOT (partial OR fragment OR hypothetical OR predicted OR uncharacterized OR␣
 ↪DAP-seq OR "transcription factor" OR DNA-binding OR microtom )
'''

ARABIDOPSIS_QUERY = '''
   "Arabidopsis thaliana"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
'''

SOLANUM_MELONGENA = '''
    "Solanum melongena"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
    '''

TUBEROSUM_QUERY = '''
    "Solanum tuberosum"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
    '''

PHYSALIS_QUERY = '''
  "Physalis peruviana"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
      '''

# ============================================================
# DOWNLOAD NCBI
# ============================================================
```

```python
def baixar_proteinas_ncbi(query, output_fasta, max_records=35000):
    print("\n================================")
    print("DOWNLOAD NCBI: ",output_fasta)
    print("================================")
    handle = Entrez.esearch(
        db="protein",
        term=query,
        retmax=max_records
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Proteínas encontradas: {len(ids)}")
    if len(ids) == 0:
        return
    handle = Entrez.efetch(
        db="protein",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")

def baixar_transcriptoma(output_fasta, query):
    print("\n================================")
    print("DOWNLOAD TRANSCRIPTOMA: ",output_fasta)
    print("================================")
    handle = Entrez.esearch(
        db="nucleotide",
        term=query,
        retmax=50000
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Transcritos encontrados: {len(ids)}")
    handle = Entrez.efetch(
        db="nucleotide",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")

#Dados de Entrada - baixar arquivos FASTA das sequências de proteínas
```

```
POSITIVE_FASTA = f"{OUTPUT_DIR}/positive.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative.fasta"


print("\n============ FINALIZADO CONFIGURAÇÃO DE PARÂMETROS ============\n")
```

```
============ FINALIZADO CONFIGURAÇÃO DE PARÂMETROS ============
```

[3]:
```
inicio = datetime.now()
print("\n\nInciando download da classe positiva: ", inicio.strftime("%H:%M:%S"))
baixar_proteinas_ncbi(POSITIVE_QUERY,POSITIVE_FASTA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando download da classe positiva:  10:32:10

================================
DOWNLOAD NCBI:  pipelineAG/positive.fasta
================================
Proteínas encontradas: 8914
Arquivo salvo: pipelineAG/positive.fasta


Finalizado em: 10:33:03

Tempo decorrido: 0:00:53.013381
```

[4]:
```
inicio = datetime.now()
print("\n\nInciando download da classe negativa: ", inicio.strftime("%H:%M:%S"))
baixar_proteinas_ncbi(NEGATIVE_QUERY,NEGATIVE_FASTA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\n\nArquivos salvos na pasta: ",OUTPUT_DIR)
print("============ FIM COLETA DE DADOS TREINO/VALIDAÇÃO ============")
```

```
Inciando download da classe negativa:  10:33:03

================================
DOWNLOAD NCBI:  pipelineAG/negative.fasta
================================
```

```
Proteínas encontradas: 35000
Arquivo salvo: pipelineAG/negative.fasta


Finalizado em: 10:34:03

Tempo decorrido: 0:01:00.093673


Arquivos salvos na pasta:  pipelineAG
============ FIM COLETA DE DADOS TREINO/VALIDAÇÃO ============
```

[3]:
```
#Dados para aplicação - baixar transcriptomas - RNA-Seq
transcriptoma_arabidopsis_fasta = (f"{OUTPUT_DIR}/arabidopsis_transcriptoma.
 ↪fasta")
transcriptoma_melongena_fasta = (f"{OUTPUT_DIR}/melongena_transcriptoma.fasta")
transcriptoma_tuberosum_fasta = (f"{OUTPUT_DIR}/tuberosum_transcriptoma.fasta")
transcriptoma_physalis_fasta = (f"{OUTPUT_DIR}/physalis_transcriptoma.fasta")
```

[6]:
```
print("\n\nInciando downaload de dados de transcriptmas")
inicio = datetime.now()
print("\n\nInciando download ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_arabidopsis_fasta, ARABIDOPSIS_QUERY)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando downaload de dados de transcriptmas


Inciando download ARABIDOPSIS:  10:34:03

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAG/arabidopsis_transcriptoma.fasta
================================
Transcritos encontrados: 49951
Arquivo salvo: pipelineAG/arabidopsis_transcriptoma.fasta


Finalizado em: 10:34:46

Tempo decorrido: 0:00:43.022369
```

[8]:
```
inicio = datetime.now()
print("\n\nInciando download PHYSALIS: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_physalis_fasta, PHYSALIS_QUERY)
```

```
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando download PHYSALIS:  10:35:39

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAG/physalis_transcriptoma.fasta
================================
Transcritos encontrados: 21702
Arquivo salvo: pipelineAG/physalis_transcriptoma.fasta


Finalizado em: 10:36:21

Tempo decorrido: 0:00:42.376873
```

[9]:
```
inicio = datetime.now()
print("\n\nInciando download MELONGENA: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_melongena_fasta, SOLANUM_MELONGENA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando download MELONGENA:  10:36:21

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAG/melongena_transcriptoma.fasta
================================
Transcritos encontrados: 8555
Arquivo salvo: pipelineAG/melongena_transcriptoma.fasta


Finalizado em: 10:36:59

Tempo decorrido: 0:00:38.041127
```

[10]:
```
inicio = datetime.now()
print("\n\nInciando download TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_tuberosum_fasta, TUBEROSUM_QUERY)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
print("\n\n\nArquivos salvos na pasta: ",OUTPUT_DIR)
print("============ FIM COLETA DE DADOS APLICAÇÃO ============")
```

```
Inciando download TUBEROSUM:  10:36:59

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAG/tuberosum_transcriptoma.fasta
================================
Transcritos encontrados: 20832
Arquivo salvo: pipelineAG/tuberosum_transcriptoma.fasta


Finalizado em: 10:37:42

Tempo decorrido: 0:00:43.120825



Arquivos salvos na pasta:  pipelineAG
============ FIM COLETA DE DADOS APLICAÇÃO ============
```

[4]:
```
'''
Tratamento dos dados
    - Criar Dataframes com os conjuntos de dados de treino e validação
    - Executar CD-Hit para eliminar sequencias muitos semelhantes
    - Extração de atributos (features)

'''

# ============================================================
# AAC
# ============================================================
# composição de aminoácidos - transformar a proteína em dados numéricos
# conta a frequência de cada aminoácido em uma sequência de proteína
# como são 20 aminoácidos possíveis, irá gerar 20 features com seus percentuais
# com essa frequência consegue ter um parâmetro para distinguir as proteínas
# por exemplo proteínas de resistência tem geralmente certa frequência de␣
 ↪aminoácidos
# mas ignora a ordem - por isso a sequência de dipeptídeos irá incrementar o␣
 ↪modelo considerando a ordem
# sequência de 20 aminoácidos
AMINO_ACIDOS = list("ACDEFGHIKLMNPQRSTVWY")

motifs = {
    # ==========================
```

```
    # NLR (CNL/TNL)
    # ==========================
    "P_LOOP": r"[AGST].{3,5}GK[TS]",
    "KINASE2": r"[ALIVMFYW]{3,5}DD[VILW][WFY]",
    "GLPL": r"G[LVI][PAST][LIVAF]",
    "MHD": r"[MVIL][HN][DE][VILAST]",
    "RNBS_A": r"F.DL.{0,3}AW",
    "RNBS_B": r"G[SA].{2,6}T",
    "RNBS_C": r"Y.[VIL].{1,3}[LS]",
    "RNBS_D": r"C.{2,8}P",

    # ==========================
    # Protein Kinase (KIN/RLK)
    # ==========================
    "HRD": r"HRD[LIVMFY]",
    "DFG": r"DFG",

    # ==========================
    # LRR (RLP/RLK)
    # ==========================
    "LRR1": r"L..L..L..N",
    "LRR2": r"L[A-Z]{2}L[A-Z]{2}L[A-Z]{2}[NCT]"

}

def calcular_aac(seq):
    seq = seq.upper()
    total = len(seq)
    if total == 0:
        return [0] * len(AMINO_ACIDOS)
    counts = Counter(seq)
    return [
        counts.get(aa, 0) / total
        for aa in AMINO_ACIDOS
    ]

# ============================================================
# DIPEPTÍDEOS
# ============================================================
'''
encontrar pares de aminoácidos consectíveis em uma sequência protéica ␣
 ↪-sequência de dois aminoácidos
considera a ordem dos aminoácidos
há certos padrões comuns para
para 20 aminoácidos, há possibilidade de 20*20 = 400 dipeptídeos - features
'''
```

```
DIPEPTIDES = [
    a + b
    for a in AMINO_ACIDOS
    for b in AMINO_ACIDOS
]

def calcular_dipeptideos(seq):
    seq = seq.upper()
    total = len(seq) - 1
    if total <= 0:
        return [0] * len(DIPEPTIDES)
    counts = Counter([
        seq[i:i+2]
        for i in range(total)
    ])
    return [
        counts.get(dp, 0) / total
        for dp in DIPEPTIDES
    ]

def localizar_motivos(seq):
    posicoes = {}
    for nome, pattern in motifs.items():
        match = re.search(pattern, seq)

        if match:
            posicoes[nome] = match.start()
        else:
            posicoes[nome] = None

    return posicoes


def calcular_distancias(posicoes):

    def distancia(a, b):
        if posicoes[a] is None or posicoes[b] is None:
            return -1
        return posicoes[b] - posicoes[a]

    return {
        "dist_ploop_kinase2": distancia("P_LOOP", "KINASE2"),
        "dist_kinase2_glpl": distancia("KINASE2", "GLPL"),
        "dist_glpl_mhd": distancia("GLPL", "MHD")
    }

def contar_motivos(posicoes):
```

```
    return sum(
        1 for v in posicoes.values()
        if v is not None
    )

def extrair_motifs(seq):

    return [
        int(bool(re.search(pattern, seq)))
        for pattern in motifs.values()
    ]



# ============================================================
# FEATURES - atributos
# ============================================================
#calculando features
def extrair_features_proteina(seq):
    seq = re.sub(r"[^ACDEFGHIKLMNPQRSTVWY]", "", seq)
    if len(seq) < 50:
        return None
    features = []
    features.append(len(seq)) # 1 - comprimento da cadeia
    features.extend(calcular_aac(seq)) # 20 composição de aminoácidos - frequência de cada um na sequência
    features.extend(calcular_dipeptideos(seq)) #400 possibilidades - considera a ordem dos aminoácidos para buscar os mais frequêntes em proteínas de resistência

    features.extend(extrair_motifs(seq))
    # posições dos motivos
    posicoes = localizar_motivos(seq)
    # número de motivos encontrados
    num_motifs_detectados = contar_motivos(posicoes)
    # distâncias entre motivos
    distancias = calcular_distancias(posicoes)

    features.append(num_motifs_detectados)
    protein_length = len(seq)

    features.append(distancias["dist_ploop_kinase2"])
    features.append(distancias["dist_kinase2_glpl"])
    features.append(distancias["dist_glpl_mhd"])

    features.append(
        distancias["dist_ploop_kinase2"]/protein_length
```

```
        if distancias["dist_ploop_kinase2"] != -1 else -1
    )
    features.append(
        distancias["dist_kinase2_glpl"]/protein_length
        if distancias["dist_kinase2_glpl"] != -1 else -1
    )
    features.append(
        distancias["dist_glpl_mhd"]/protein_length
        if distancias["dist_glpl_mhd"] != -1 else -1
    )

    features.append(int(posicoes["P_LOOP"] is not None))
    features.append(int(posicoes["KINASE2"] is not None))
    features.append(int(posicoes["GLPL"] is not None))
    features.append(int(posicoes["MHD"] is not None))

    encontrados = {}
    for nome, regex in motifs.items():
        encontrados[nome] = int( re.search(regex, seq) is not None)

    # features.append(encontrados[nome])

    num_nlr = (
        encontrados["P_LOOP"] +
        encontrados["KINASE2"] +
        encontrados["GLPL"] +
        encontrados["MHD"]
    )

    # num_kinase = (
    #     encontrados["HRD"] +
    #     encontrados["DFG"]
    # )

    # num_lrr = (
    #     encontrados["LRR1"] +
    #     encontrados["LRR2"]
    # )

    features.extend([
        num_nlr,
        # num_kinase,
        # num_lrr
    ])

    return features
```

```
# ============================================================
# DATASET
# ============================================================
motif_cols = [
    "P_LOOP",
    "KINASE2",
    "GLPL",
    "MHD",
    "RNBS_A",
    "RNBS_B",
    "RNBS_C",
    "RNBS_D",
    "HRD",
    "DFG",
    "LRR1",
    "LRR2",
    "NUM_NLR_MOTIFS"
 ]
    # "NUM_KINASE_MOTIFS",
    # "NUM_LRR_MOTIFS"

def criar_dataset(positive_fasta, negative_fasta):
    data = []
    columns = (
        ["protein_length"]
        + [f"AAC_{aa}" for aa in AMINO_ACIDOS]
        + [f"DIPEP_{dp}" for dp in DIPEPTIDES]
        + motif_cols
        + ["num_motifs_detectados"]
        + ["dist_ploop_kinase2"]
        + ["dist_kinase2_glpl"]
        + ["dist_glpl_mhd"]
        + ["dist_ploop_kinase2_norm"]
        + ["dist_kinase2_glpl_norm"]
        + ["dist_glpl_mhd_norm"]
        + ["has_ploop"]
        + ["has_kinase2"]
        + ["has_glpl"]
        + ["has_mhd"]
    )

    # POSITIVOS
    for record in SeqIO.parse(positive_fasta, "fasta"):
        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
```

```
        row = feats + [1, record.id]
        data.append(row)

    # NEGATIVOS
    for record in SeqIO.parse(negative_fasta, "fasta"):
        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
        row = feats + [0, record.id]
        data.append(row)

    df = pd.DataFrame(
        data,
        columns=columns + ["target", "id"]
    )
    print("\nTotal de Features/Atributos:",len(columns)) # quantidade
    print("\nFeatures/atributos:")
    print(columns)
    return df, columns

# CD-Hit
# chama o programa CD-HIT do sistema operacional
def executar_cdhit(
    input_fasta,
    output_fasta,
    identity=0.7
):
    cmd = [
        "cd-hit",
        "-i", input_fasta,
        "-o", output_fasta,
        "-c", str(identity),
        "-n", "5",
        "-M", "30000",
        "-T", "4"
    ]

    print("\nExecutando CD-HIT...")
    subprocess.run(cmd, check=True)
    print("CD-HIT concluído.")

def limpar(seq_id):
    seq_id = seq_id.strip()
    seq_id = seq_id.split()[0]
    return seq_id
```

```
def ler_clusters_cdhit(cluster_file):
    clusters = {}
    cluster_id = None
    with open(cluster_file) as f:
        for line in f:
            line = line.strip()
            if line.startswith(">Cluster"):
                cluster_id = line.replace(">Cluster ", "")
                clusters[cluster_id] = []
            else:
                seq_id = line.split(">")[1].split("...")[0]
                seq_id = limpar(seq_id)
                clusters[cluster_id].append(seq_id)
    return clusters

inicio = datetime.now()
print("\n\n\nInciando Tratamento dos dados: ", inicio.strftime("%H:%M:%S"))

# executa o CD-Hit nos arquivos baixados
print("\n\nInciar a execução do CD-HIT\n")
executar_cdhit(
    POSITIVE_FASTA,
    f"{OUTPUT_DIR}/positive_nr.fasta",
    identity=0.7
)

executar_cdhit(
    NEGATIVE_FASTA,
    f"{OUTPUT_DIR}/negative_nr.fasta",
    identity=0.7
)
# ========================================================
# Montar os DATASETs
# ========================================================

POSITIVE_FASTA = f"{OUTPUT_DIR}/positive_nr.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative_nr.fasta"


df, columns = criar_dataset(
    POSITIVE_FASTA,
    NEGATIVE_FASTA
)

df["id"] = df["id"].apply(limpar)
# ========================================================
# CLUSTERS
```

```
# ======================================================

clusters_pos = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/positive_nr.fasta.clstr"
)

clusters_neg = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/negative_nr.fasta.clstr"
)

cluster_map = {}

for c, seqs in clusters_pos.items():
    for s in seqs:
        cluster_map[s] = f"POS_{c}"

for c, seqs in clusters_neg.items():
    for s in seqs:
        cluster_map[s] = f"NEG_{c}"

df["cluster"] = df["id"].map(cluster_map)

print("\n Clusters ausentes: ")
print(df["cluster"].isna().sum())
df = df.dropna(subset=["cluster"])

print("\nDataset original pós CD-HIT:", df.shape)

# ======================================================
# NOVO BLOCO: BALANCEAMENTO IMPERATIVO (DOWNSAMPLING)
# ======================================================
print("\n--- Executando Balanceamento de Classes ---")
df_positivos = df[df["target"] == 1]
df_negativos = df[df["target"] == 0]

print(f"Contagem inicial -> Positivos (R-genes): {len(df_positivos)} |␣
 ↪Negativos (Background): {len(df_negativos)}")

# Se houver mais negativos do que positivos (cenário padrão), reduzimos os␣
 ↪negativos
if len(df_negativos) > len(df_positivos):
    # O sample() escolhe aleatoriamente N linhas do grupo majoritário
    df_negativos_balanceados = df_negativos.sample(n=len(df_positivos),␣
 ↪random_state=42)
    df = pd.concat([df_positivos, df_negativos_balanceados]).
 ↪reset_index(drop=True)
    print(f"Subamostragem aplicada com sucesso nos Negativos.")
```

```
else:
    print("O dataset já está balanceado ou possui mais positivos que negativos␣
 ↪(incomum). Nenhuma ação foi tomada.")

print(f"Contagem final   -> Positivos: {len(df[df['target'] == 1])} | Negativos:
 ↪ {len(df[df['target'] == 0])}")
print("Dataset Final Pronto:", df.shape)


fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
print("\n\nFim tratamento dos dados...")
```

```
Inciando Tratamento dos dados:  14:19:51


Inciar a execução do CD-HIT


Executando CD-HIT…
================================================================
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineAG/positive.fasta -o
         pipelineAG/positive_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

Started: Thu Jun 18 14:19:51 2026
================================================================
                            Output
----------------------------------------------------------------
total seq: 8914
longest and shortest : 3540 and 51
Total letters: 7801441
Sequences have been sorted

Approximated minimal memory consumption:
Sequence        : 8M
Buffer          : 4 X 11M = 45M
Table           : 2 X 65M = 130M
Miscellaneous   : 0M
Total           : 185M

Table limit with the given memory limit:
Max number of representatives: 4000000
```

```
Max number of word counting entries: 3726820522

# comparing sequences from          0  to       1485
.---------- new table with      236 representatives
# comparing sequences from       1485  to       2723
99.9%---------- new table with      192 representatives
# comparing sequences from       2723  to       3754
----------     96 remaining sequences to the next cycle
---------- new table with      164 representatives
# comparing sequences from       3658  to       4534
99.9%---------- new table with      123 representatives
# comparing sequences from       4534  to       5264
99.6%---------- new table with       63 representatives
# comparing sequences from       5264  to       5872
…---------- new table with       88 representatives
# comparing sequences from       5872  to       6379
----------    105 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       6274  to       6714
----------    102 remaining sequences to the next cycle
---------- new table with      112 representatives
# comparing sequences from       6612  to       6995
----------     45 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       6950  to       7277
----------     61 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       7216  to       7499
…---------- new table with       76 representatives
# comparing sequences from       7499  to       7734
…---------- new table with       66 representatives
# comparing sequences from       7734  to       7930
…---------- new table with       51 representatives
# comparing sequences from       7930  to       8914
…---------- new table with      248 representatives

     8914  finished       1719  clusters

Approximated maximum memory consumption: 190M
writing new database
writing clustering information
program completed !

Total CPU time 3.72
CD-HIT concluído.

Executando CD-HIT…
================================================================
```

```
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineAG/negative.fasta -o
         pipelineAG/negative_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

Started: Thu Jun 18 14:19:53 2026
================================================================
                            Output
----------------------------------------------------------------
total seq: 9999
longest and shortest : 2993 and 100
Total letters: 4204463
Sequences have been sorted

Approximated minimal memory consumption:
Sequence        : 5M
Buffer          : 4 X 11M = 44M
Table           : 2 X 65M = 131M
Miscellaneous   : 0M
Total           : 181M

Table limit with the given memory limit:
Max number of representatives: 4000000
Max number of word counting entries: 3727298858

# comparing sequences from          0  to       1666
.---------- new table with     1374 representatives
# comparing sequences from       1666  to       3054
----------    588 remaining sequences to the next cycle
---------- new table with      659 representatives
# comparing sequences from       2466  to       3721
----------    812 remaining sequences to the next cycle
---------- new table with      261 representatives
# comparing sequences from       2909  to       4090
----------    830 remaining sequences to the next cycle
---------- new table with      250 representatives
# comparing sequences from       3260  to       4383
----------    775 remaining sequences to the next cycle
---------- new table with      290 representatives
# comparing sequences from       3608  to       4673
----------    717 remaining sequences to the next cycle
---------- new table with      291 representatives
# comparing sequences from       3956  to       4963
----------    681 remaining sequences to the next cycle
---------- new table with      235 representatives
# comparing sequences from       4282  to       5234
----------    657 remaining sequences to the next cycle
---------- new table with      232 representatives
# comparing sequences from       4577  to       5480
```

```
----------    630 remaining sequences to the next cycle
---------- new table with      217 representatives
# comparing sequences from       4850  to       5708
----------    592 remaining sequences to the next cycle
---------- new table with      187 representatives
# comparing sequences from       5116  to       5929
----------    552 remaining sequences to the next cycle
---------- new table with      200 representatives
# comparing sequences from       5377  to       6147
----------    538 remaining sequences to the next cycle
---------- new table with      168 representatives
# comparing sequences from       5609  to       6340
----------    495 remaining sequences to the next cycle
---------- new table with      162 representatives
# comparing sequences from       5845  to       6537
----------    472 remaining sequences to the next cycle
---------- new table with      173 representatives
# comparing sequences from       6065  to       6720
----------    448 remaining sequences to the next cycle
---------- new table with      156 representatives
# comparing sequences from       6272  to       6893
----------    438 remaining sequences to the next cycle
---------- new table with      123 representatives
# comparing sequences from       6455  to       7045
----------    372 remaining sequences to the next cycle
---------- new table with      148 representatives
# comparing sequences from       6673  to       7227
----------    361 remaining sequences to the next cycle
---------- new table with      116 representatives
# comparing sequences from       6866  to       7388
----------    326 remaining sequences to the next cycle
---------- new table with      138 representatives
# comparing sequences from       7062  to       7551
----------    326 remaining sequences to the next cycle
---------- new table with      114 representatives
# comparing sequences from       7225  to       7687
----------    295 remaining sequences to the next cycle
---------- new table with      111 representatives
# comparing sequences from       7392  to       7826
----------    266 remaining sequences to the next cycle
---------- new table with      118 representatives
# comparing sequences from       7560  to       7966
----------    246 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       7720  to       8099
----------    215 remaining sequences to the next cycle
---------- new table with      103 representatives
# comparing sequences from       7884  to       8236
```

```
----------    212 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8024  to       8353
----------    173 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8180  to       8483
----------    169 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8314  to       8594
----------    128 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8466  to       8721
----------     10 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8711  to       8925
----------     66 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8859  to       9049
----------     53 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8996  to       9163
…---------- new table with       62 representatives
# comparing sequences from       9163  to       9999
…---------- new table with      405 representatives

     9999  finished       7093  clusters

Approximated maximum memory consumption: 194M
writing new database
writing clustering information
program completed !

Total CPU time 1.72
CD-HIT concluído.

Total de Features/Atributos: 445

Features/atributos:
['protein_length', 'AAC_A', 'AAC_C', 'AAC_D', 'AAC_E', 'AAC_F', 'AAC_G',
'AAC_H', 'AAC_I', 'AAC_K', 'AAC_L', 'AAC_M', 'AAC_N', 'AAC_P', 'AAC_Q', 'AAC_R',
'AAC_S', 'AAC_T', 'AAC_V', 'AAC_W', 'AAC_Y', 'DIPEP_AA', 'DIPEP_AC', 'DIPEP_AD',
'DIPEP_AE', 'DIPEP_AF', 'DIPEP_AG', 'DIPEP_AH', 'DIPEP_AI', 'DIPEP_AK',
'DIPEP_AL', 'DIPEP_AM', 'DIPEP_AN', 'DIPEP_AP', 'DIPEP_AQ', 'DIPEP_AR',
'DIPEP_AS', 'DIPEP_AT', 'DIPEP_AV', 'DIPEP_AW', 'DIPEP_AY', 'DIPEP_CA',
'DIPEP_CC', 'DIPEP_CD', 'DIPEP_CE', 'DIPEP_CF', 'DIPEP_CG', 'DIPEP_CH',
'DIPEP_CI', 'DIPEP_CK', 'DIPEP_CL', 'DIPEP_CM', 'DIPEP_CN', 'DIPEP_CP',
'DIPEP_CQ', 'DIPEP_CR', 'DIPEP_CS', 'DIPEP_CT', 'DIPEP_CV', 'DIPEP_CW',
'DIPEP_CY', 'DIPEP_DA', 'DIPEP_DC', 'DIPEP_DD', 'DIPEP_DE', 'DIPEP_DF',
```

'DIPEP_DG', 'DIPEP_DH', 'DIPEP_DI', 'DIPEP_DK', 'DIPEP_DL', 'DIPEP_DM',
'DIPEP_DN', 'DIPEP_DP', 'DIPEP_DQ', 'DIPEP_DR', 'DIPEP_DS', 'DIPEP_DT',
'DIPEP_DV', 'DIPEP_DW', 'DIPEP_DY', 'DIPEP_EA', 'DIPEP_EC', 'DIPEP_ED',
'DIPEP_EE', 'DIPEP_EF', 'DIPEP_EG', 'DIPEP_EH', 'DIPEP_EI', 'DIPEP_EK',
'DIPEP_EL', 'DIPEP_EM', 'DIPEP_EN', 'DIPEP_EP', 'DIPEP_EQ', 'DIPEP_ER',
'DIPEP_ES', 'DIPEP_ET', 'DIPEP_EV', 'DIPEP_EW', 'DIPEP_EY', 'DIPEP_FA',
'DIPEP_FC', 'DIPEP_FD', 'DIPEP_FE', 'DIPEP_FF', 'DIPEP_FG', 'DIPEP_FH',
'DIPEP_FI', 'DIPEP_FK', 'DIPEP_FL', 'DIPEP_FM', 'DIPEP_FN', 'DIPEP_FP',
'DIPEP_FQ', 'DIPEP_FR', 'DIPEP_FS', 'DIPEP_FT', 'DIPEP_FV', 'DIPEP_FW',
'DIPEP_FY', 'DIPEP_GA', 'DIPEP_GC', 'DIPEP_GD', 'DIPEP_GE', 'DIPEP_GF',
'DIPEP_GG', 'DIPEP_GH', 'DIPEP_GI', 'DIPEP_GK', 'DIPEP_GL', 'DIPEP_GM',
'DIPEP_GN', 'DIPEP_GP', 'DIPEP_GQ', 'DIPEP_GR', 'DIPEP_GS', 'DIPEP_GT',
'DIPEP_GV', 'DIPEP_GW', 'DIPEP_GY', 'DIPEP_HA', 'DIPEP_HC', 'DIPEP_HD',
'DIPEP_HE', 'DIPEP_HF', 'DIPEP_HG', 'DIPEP_HH', 'DIPEP_HI', 'DIPEP_HK',
'DIPEP_HL', 'DIPEP_HM', 'DIPEP_HN', 'DIPEP_HP', 'DIPEP_HQ', 'DIPEP_HR',
'DIPEP_HS', 'DIPEP_HT', 'DIPEP_HV', 'DIPEP_HW', 'DIPEP_HY', 'DIPEP_IA',
'DIPEP_IC', 'DIPEP_ID', 'DIPEP_IE', 'DIPEP_IF', 'DIPEP_IG', 'DIPEP_IH',
'DIPEP_II', 'DIPEP_IK', 'DIPEP_IL', 'DIPEP_IM', 'DIPEP_IN', 'DIPEP_IP',
'DIPEP_IQ', 'DIPEP_IR', 'DIPEP_IS', 'DIPEP_IT', 'DIPEP_IV', 'DIPEP_IW',
'DIPEP_IY', 'DIPEP_KA', 'DIPEP_KC', 'DIPEP_KD', 'DIPEP_KE', 'DIPEP_KF',
'DIPEP_KG', 'DIPEP_KH', 'DIPEP_KI', 'DIPEP_KK', 'DIPEP_KL', 'DIPEP_KM',
'DIPEP_KN', 'DIPEP_KP', 'DIPEP_KQ', 'DIPEP_KR', 'DIPEP_KS', 'DIPEP_KT',
'DIPEP_KV', 'DIPEP_KW', 'DIPEP_KY', 'DIPEP_LA', 'DIPEP_LC', 'DIPEP_LD',
'DIPEP_LE', 'DIPEP_LF', 'DIPEP_LG', 'DIPEP_LH', 'DIPEP_LI', 'DIPEP_LK',
'DIPEP_LL', 'DIPEP_LM', 'DIPEP_LN', 'DIPEP_LP', 'DIPEP_LQ', 'DIPEP_LR',
'DIPEP_LS', 'DIPEP_LT', 'DIPEP_LV', 'DIPEP_LW', 'DIPEP_LY', 'DIPEP_MA',
'DIPEP_MC', 'DIPEP_MD', 'DIPEP_ME', 'DIPEP_MF', 'DIPEP_MG', 'DIPEP_MH',
'DIPEP_MI', 'DIPEP_MK', 'DIPEP_ML', 'DIPEP_MM', 'DIPEP_MN', 'DIPEP_MP',
'DIPEP_MQ', 'DIPEP_MR', 'DIPEP_MS', 'DIPEP_MT', 'DIPEP_MV', 'DIPEP_MW',
'DIPEP_MY', 'DIPEP_NA', 'DIPEP_NC', 'DIPEP_ND', 'DIPEP_NE', 'DIPEP_NF',
'DIPEP_NG', 'DIPEP_NH', 'DIPEP_NI', 'DIPEP_NK', 'DIPEP_NL', 'DIPEP_NM',
'DIPEP_NN', 'DIPEP_NP', 'DIPEP_NQ', 'DIPEP_NR', 'DIPEP_NS', 'DIPEP_NT',
'DIPEP_NV', 'DIPEP_NW', 'DIPEP_NY', 'DIPEP_PA', 'DIPEP_PC', 'DIPEP_PD',
'DIPEP_PE', 'DIPEP_PF', 'DIPEP_PG', 'DIPEP_PH', 'DIPEP_PI', 'DIPEP_PK',
'DIPEP_PL', 'DIPEP_PM', 'DIPEP_PN', 'DIPEP_PP', 'DIPEP_PQ', 'DIPEP_PR',
'DIPEP_PS', 'DIPEP_PT', 'DIPEP_PV', 'DIPEP_PW', 'DIPEP_PY', 'DIPEP_QA',
'DIPEP_QC', 'DIPEP_QD', 'DIPEP_QE', 'DIPEP_QF', 'DIPEP_QG', 'DIPEP_QH',
'DIPEP_QI', 'DIPEP_QK', 'DIPEP_QL', 'DIPEP_QM', 'DIPEP_QN', 'DIPEP_QP',
'DIPEP_QQ', 'DIPEP_QR', 'DIPEP_QS', 'DIPEP_QT', 'DIPEP_QV', 'DIPEP_QW',
'DIPEP_QY', 'DIPEP_RA', 'DIPEP_RC', 'DIPEP_RD', 'DIPEP_RE', 'DIPEP_RF',
'DIPEP_RG', 'DIPEP_RH', 'DIPEP_RI', 'DIPEP_RK', 'DIPEP_RL', 'DIPEP_RM',
'DIPEP_RN', 'DIPEP_RP', 'DIPEP_RQ', 'DIPEP_RR', 'DIPEP_RS', 'DIPEP_RT',
'DIPEP_RV', 'DIPEP_RW', 'DIPEP_RY', 'DIPEP_SA', 'DIPEP_SC', 'DIPEP_SD',
'DIPEP_SE', 'DIPEP_SF', 'DIPEP_SG', 'DIPEP_SH', 'DIPEP_SI', 'DIPEP_SK',
'DIPEP_SL', 'DIPEP_SM', 'DIPEP_SN', 'DIPEP_SP', 'DIPEP_SQ', 'DIPEP_SR',
'DIPEP_SS', 'DIPEP_ST', 'DIPEP_SV', 'DIPEP_SW', 'DIPEP_SY', 'DIPEP_TA',
'DIPEP_TC', 'DIPEP_TD', 'DIPEP_TE', 'DIPEP_TF', 'DIPEP_TG', 'DIPEP_TH',
'DIPEP_TI', 'DIPEP_TK', 'DIPEP_TL', 'DIPEP_TM', 'DIPEP_TN', 'DIPEP_TP',

```
'DIPEP_TQ', 'DIPEP_TR', 'DIPEP_TS', 'DIPEP_TT', 'DIPEP_TV', 'DIPEP_TW',
'DIPEP_TY', 'DIPEP_VA', 'DIPEP_VC', 'DIPEP_VD', 'DIPEP_VE', 'DIPEP_VF',
'DIPEP_VG', 'DIPEP_VH', 'DIPEP_VI', 'DIPEP_VK', 'DIPEP_VL', 'DIPEP_VM',
'DIPEP_VN', 'DIPEP_VP', 'DIPEP_VQ', 'DIPEP_VR', 'DIPEP_VS', 'DIPEP_VT',
'DIPEP_VV', 'DIPEP_VW', 'DIPEP_VY', 'DIPEP_WA', 'DIPEP_WC', 'DIPEP_WD',
'DIPEP_WE', 'DIPEP_WF', 'DIPEP_WG', 'DIPEP_WH', 'DIPEP_WI', 'DIPEP_WK',
'DIPEP_WL', 'DIPEP_WM', 'DIPEP_WN', 'DIPEP_WP', 'DIPEP_WQ', 'DIPEP_WR',
'DIPEP_WS', 'DIPEP_WT', 'DIPEP_WV', 'DIPEP_WW', 'DIPEP_WY', 'DIPEP_YA',
'DIPEP_YC', 'DIPEP_YD', 'DIPEP_YE', 'DIPEP_YF', 'DIPEP_YG', 'DIPEP_YH',
'DIPEP_YI', 'DIPEP_YK', 'DIPEP_YL', 'DIPEP_YM', 'DIPEP_YN', 'DIPEP_YP',
'DIPEP_YQ', 'DIPEP_YR', 'DIPEP_YS', 'DIPEP_YT', 'DIPEP_YV', 'DIPEP_YW',
'DIPEP_YY', 'P_LOOP', 'KINASE2', 'GLPL', 'MHD', 'RNBS_A', 'RNBS_B', 'RNBS_C',
'RNBS_D', 'HRD', 'DFG', 'LRR1', 'LRR2', 'NUM_NLR_MOTIFS',
'num_motifs_detectados', 'dist_ploop_kinase2', 'dist_kinase2_glpl',
'dist_glpl_mhd', 'dist_ploop_kinase2_norm', 'dist_kinase2_glpl_norm',
'dist_glpl_mhd_norm', 'has_ploop', 'has_kinase2', 'has_glpl', 'has_mhd']

 Clusters ausentes:
168

Dataset original pós CD-HIT: (8644, 448)

--- Executando Balanceamento de Classes ---
Contagem inicial -> Positivos (R-genes): 1700 | Negativos (Background): 6944
Subamostragem aplicada com sucesso nos Negativos.
Contagem final   -> Positivos: 1700 | Negativos: 1700
Dataset Final Pronto: (3400, 448)


Finalizado em: 14:19:55

Tempo decorrido: 0:00:03.775919


Fim tratamento dos dados…
```

[5]:
```
'''
Funções para treino e validação dos modelos

'''

def validacao_cruzada(nome, model, X_train, y_train, groups_train):

    cv = StratifiedGroupKFold(
        n_splits=10, #5
        shuffle=True,
        random_state=42
```

```
    )
    mcc_scorer = make_scorer(matthews_corrcoef)
    scoring_dict = {
        'recall': 'recall',
        'roc_auc': 'roc_auc',
        'mcc': mcc_scorer,
        'accuracy': 'accuracy',
        'precision': 'precision',
        'f1': 'f1'
    }
    scores = cross_validate(
        model,
        X_train,
        y_train,
        groups=groups_train,
        cv=cv,
        scoring=scoring_dict,
        n_jobs=-1
    )
    print("\n\n=========================\n\n")
    print("\n\nValidação: ",nome)
    print("\n\nRecall:")
    print(scores['test_recall'])
    print("\n\nMCC:")
    print(scores['test_mcc'])
    print("\n\nACC:")
    print(scores['test_accuracy'])
    print("\n\nROC:")
    print(scores['test_roc_auc'])
    print("\n\nF1:")
    print(scores['test_f1'])
    print("\n\n=========================\n\n")

    print(f"Finalizada a validação cruzada para: {nome}")
    return scores

# para melhor separar os conjuntos de treino e teste considerando os␣
 ↪agrupamentos feitos no CD-HIT para sequências muito parecidas
def split_por_homologia(df):
    print(df.head(30))
    groups = df["cluster"]
    splitter = GroupShuffleSplit(
        n_splits=1,
        test_size=0.2,
        random_state=42
    )
    # cv  = StratifiedGroupKFold(
```

```
    #     n_splits = 10,
    #     shuffle=True,
    #     random_state = 42
    # )

    train_idx, test_idx = next(
        splitter.split(df, groups=groups)
        # cv.split(df, df["target"], groups)
    )
    train_df = df.iloc[train_idx]
    test_df = df.iloc[test_idx]
    return train_df, test_df

def calcular_e_salvar_shap(nome, model, X_test, feature_names, output_dir):
    """
    Calcula os SHAP values para modelos de árvore e salva os gráficos␣
 ↪explicativos.
    """
    print(f"\n[SHAP] Inicializando análise de interpretabilidade para: {nome}...
 ↪")

    # 1. Inicializa o explicador otimizado para árvores
    explainer = shap.TreeExplainer(model)

    # 2. Calcula os SHAP values para o conjunto de teste
    shap_values = explainer.shap_values(X_test)

    # 3. Tratamento de formato (Scikit-Learn Random Forest vs XGBoost)
    # O Random Forest do sklearn retorna uma lista contendo [valores_classe_0,␣
 ↪valores_classe_1]
    # O XGBoost retorna diretamente a matriz de SHAP values para a classe alvo
    if isinstance(shap_values, list):
        # Selecionamos o índice 1 (classe positiva: proteínas de resistência/
 ↪alvos)
        shap_values_pos = shap_values[1]
    elif isinstance(shap_values, np.ndarray) and len(shap_values.shape) == 3:
        # Tratamento para versões específicas do SHAP que geram arrays 3D
        shap_values_pos = shap_values[:, :, 1]
    else:
        shap_values_pos = shap_values

    # 4. Gráfico 1: Summary Plot (Beeswarm)
    # Mostra o impacto biológico de cada feature (ex: se alta frequência␣
 ↪aumenta ou diminui a predição)
    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values_pos,
```

```
        X_test,
        feature_names=feature_names,
        max_display=20,  # Foco nas top 20 características
        show=False
    )
    # plt.title(f"SHAP Summary Plot (Top 20) - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_summary.png", bbox_inches="tight",␣
↪dpi=300)
    plt.close()

    # 5. Gráfico 2: Bar Plot (Importância Global)
    # Mostra a magnitude média do impacto de cada feature de forma absoluta
    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values_pos,
        X_test,
        feature_names=feature_names,
        plot_type="bar",
        max_display=20,
        show=False
    )
    # plt.title(f"SHAP Global Feature Importance - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_bar.png", bbox_inches="tight",␣
↪dpi=300)
    plt.close()

    print(f"[SHAP] Gráficos salvos com sucesso em '{output_dir}/'!")


def calcular_shap_svm(nome, model, feature_names,  X_train, X_test, y_test,␣
↪output_dir):
    # ======================================================
    # IMPORTÂNCIA POR PERMUTAÇÃO PARA O SVM (Novo)
    # ======================================================
    print("\n[SVM] Calculando a importância por permutação dos atributos...")

    # Calcula a permutação no X_test (pode usar X_train se quiser focar no␣
↪ajuste ao treino)
    result_svm = permutation_importance(model, X_test, y_test, n_repeats=5,␣
↪random_state=42, n_jobs=-1)

    # Monta o DataFrame com os resultados
    importancia_svm = pd.DataFrame({
        'feature': columns,
        'importance_mean': result_svm.importances_mean
```

```
    }).sort_values('importance_mean', ascending=False)

    print("\nTop 20 features mais importantes para o SVM:")
    print(importancia_svm.head(20))

    # Gera o gráfico de barras igual ao do XGBoost
    top_svm = importancia_svm.head(20)
    plt.figure(figsize=(10,8))
    plt.barh(top_svm["feature"][::-1], top_svm["importance_mean"][::-1],␣
↪color='seagreen')
    plt.xlabel("Queda no Desempenho (Média)")
    plt.ylabel("Feature / Atributo")
    # plt.title("Top 20 Features - SVM Permutation Importance")
    plt.tight_layout()
    plt.savefig(f"{output_dir}/svm_importancia_permutacao.png",␣
↪bbox_inches="tight", dpi=300)
    plt.close()

    idx_back = np.random.choice(
        len(X_train),
        min(100,len(X_train)),
        replace = False
    )
    back = X_train[idx_back]

    idx_test = np.random.choice(
        len(X_test),
        min(200,len(X_test)),
        replace = False
    )
    x_test_sample = X_test[idx_test]

    print('\nCriando explainer shap...\n')
    explainer = shap.Explainer(model.predict, back)
    shap_values = explainer(x_test_sample, max_evals=1000)
    print("\nShap SHAP:",shap_values.values.shape)

    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values.values,
        x_test_sample,
        feature_names=feature_names,
      #  plot_type="bar",
        max_display=20,
        show=False
    )
    # plt.title(f"SHAP Global Feature Importance - {nome}", fontsize=14, pad=20)
```

```
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_bar_summary.png",␣
 ↪bbox_inches="tight", dpi=300)
    plt.close()

    print(f"[SVM] Gráfico salvo com sucesso em '{output_dir}/
 ↪svm_importancia_permutacao.png'!")

# ============================================================
# TREINAMENTO - treina os modelos
# ============================================================

def avaliar_modelo(nome, model, X_train, y_train, X_test, y_test):
    print("\n============================================================")
    print(nome)
    print("============================================================")
    model.fit(X_train, y_train)

    probs = model.predict_proba(X_test)[:,1]
    preds = (probs >= 0.5).astype(int)
    print(classification_report(y_test, preds))
    roc = roc_auc_score(y_test, probs)
    pr = average_precision_score(y_test, probs)
    mcc = matthews_corrcoef(y_test, preds)
    f1 = f1_score(y_test, preds)
    precision = precision_score(y_test, preds)
    recall = recall_score(y_test, preds)
    bal_acc = balanced_accuracy_score(y_test, preds)
    acc = accuracy_score(y_test, preds)

    print("ROC-AUC          :", roc)
    print("PR-AUC           :", pr)
    print("MCC              :", mcc)
    print("F1               :", f1)
    print("Precision        :", precision)
    print("Recall           :", recall)
    print("Balanced Accuracy:", bal_acc)
    print("Standard Accuracy:", acc)

    # ========================================================
    # MATRIZ DE CONFUSÃO
    # ========================================================
    cm = confusion_matrix(y_test, preds)
    plt.figure(figsize=(5,5))
    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=["Negativo", "Positivo"]
```

```
    )
    disp.plot(cmap="Blues")
    # plt.title(f"Matriz de Confusão - {nome}")
    plt.savefig(
        f"{OUTPUT_DIR}/{nome}_matriz_confusao.png",
        bbox_inches="tight"
    )
    plt.close()
    # ========================================================
    # CURVA ROC
    # ========================================================
    fpr, tpr, _ = roc_curve(y_test, probs)
    roc_auc = auc(fpr, tpr)
    plt.figure(figsize=(6,6))
    plt.plot(
        fpr,
        tpr,
        label=f"AUC = {roc_auc:.4f}"
    )
    plt.plot([0,1], [0,1], linestyle="--")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    # plt.title(f"Curva ROC - {nome}")
    plt.legend(loc="lower right")
    plt.savefig(
        f"{OUTPUT_DIR}/{nome}_roc.png",
        bbox_inches="tight"
    )
    plt.close()

    return {
        "Modelo": nome,
        "ROC_AUC": roc,
        "F1": f1,
        "Precision": precision,
        "Recall": recall,
        "Balanced_Accuracy": bal_acc,
        "Accuracy": acc,
        "MCC":mcc
    }, model
```

[6]:
```
'''
Criação dos modelos
'''
inicio_treinamento = datetime.now()
print("\n\n\nInciando Treinamento dos modelos: ", inicio_treinamento.
 ↪strftime("%H:%M:%S"))
```

```
train_df, test_df = split_por_homologia(df)
X_train = train_df[columns]
y_train = train_df["target"]
X_test = test_df[columns]
y_test = test_df["target"]
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

rf = RandomForestClassifier(
    n_estimators=1000,
    max_depth = None,
    min_samples_split=5,
    max_features=0.2,
    random_state=42,
    class_weight="balanced",
    n_jobs = 1
)

xgb = XGBClassifier(
    n_estimators=1200,
    max_depth=7,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42,
    min_child_weight=3,
    reg_alpha=0.5,
    reg_lambda=2,
    scale_pos_weight=1.0
)

svm = SVC(
    probability=True,
    kernel="rbf",
    class_weight="balanced",
    random_state=42,
    C=2,
    gamma='scale'
)
```

```
Inciando Treinamento dos modelos:  14:20:03
    protein_length     AAC_A     AAC_C     AAC_D     AAC_E     AAC_F  \
```

0	231	0.090909	0.012987	0.069264	0.077922	0.051948
1	704	0.049716	0.007102	0.080966	0.117898	0.041193
2	971	0.045314	0.017508	0.054583	0.071061	0.036045
3	323	0.046440	0.027864	0.052632	0.080495	0.046440
4	929	0.034446	0.034446	0.046286	0.069968	0.053821
5	709	0.053597	0.014104	0.049365	0.071932	0.043724
6	999	0.053053	0.018018	0.070070	0.050050	0.036036
7	1071	0.056956	0.020542	0.059757	0.038282	0.043884
8	1050	0.043810	0.022857	0.053333	0.080952	0.041905
9	904	0.042035	0.022124	0.032080	0.055310	0.056416
10	1152	0.065972	0.015625	0.041667	0.046875	0.030382
11	949	0.033720	0.034773	0.052687	0.067439	0.041096
12	272	0.055147	0.033088	0.040441	0.084559	0.040441
13	428	0.049065	0.023364	0.070093	0.070093	0.051402
14	253	0.059289	0.019763	0.071146	0.051383	0.039526
15	886	0.038375	0.024831	0.060948	0.055305	0.050790
16	910	0.052747	0.017582	0.060440	0.065934	0.052747
17	879	0.035267	0.018203	0.052332	0.073948	0.046644
18	239	0.041841	0.020921	0.083682	0.050209	0.046025
19	126	0.055556	0.031746	0.055556	0.055556	0.039683
20	104	0.028846	0.038462	0.038462	0.067308	0.028846
21	105	0.057143	0.019048	0.076190	0.019048	0.057143
22	716	0.044693	0.023743	0.061453	0.067039	0.046089
23	196	0.061224	0.010204	0.040816	0.091837	0.035714
24	122	0.040984	0.008197	0.081967	0.073770	0.040984
25	1289	0.044220	0.017843	0.061288	0.074476	0.046548
26	846	0.034279	0.017730	0.063830	0.069740	0.047281
27	883	0.055493	0.019253	0.057758	0.070215	0.048698
28	906	0.052980	0.020971	0.055188	0.073951	0.049669
29	343	0.084548	0.011662	0.061224	0.064140	0.037901

	AAC_G	AAC_H	AAC_I	AAC_K	…	dist_ploop_kinase2_norm	\
0	0.077922	0.038961	0.043290	0.082251	…	-1.000000	
1	0.048295	0.009943	0.065341	0.117898	…	-1.000000	
2	0.055613	0.024717	0.057673	0.061792	…	0.094748	
3	0.065015	0.024768	0.061920	0.052632	…	-1.000000	
4	0.051668	0.027987	0.071044	0.059203	…	-1.000000	
5	0.063470	0.021157	0.063470	0.077574	…	-1.000000	
6	0.069069	0.021021	0.043043	0.077077	…	-1.000000	
7	0.090570	0.023343	0.040149	0.059757	…	-1.000000	
8	0.048571	0.024762	0.065714	0.058095	…	-0.023810	
9	0.059735	0.028761	0.065265	0.074115	…	-1.000000	
10	0.110243	0.026910	0.039062	0.046007	…	-1.000000	
11	0.049526	0.025290	0.060063	0.052687	…	0.089568	
12	0.044118	0.029412	0.047794	0.066176	…	-1.000000	
13	0.053738	0.011682	0.072430	0.077103	…	-1.000000	
14	0.063241	0.027668	0.075099	0.102767	…	0.280632	
15	0.053047	0.027088	0.067720	0.082393	…	0.106095	

```
16  0.040659  0.021978  0.071429  0.086813  …                0.100000
17  0.039818  0.036405  0.069397  0.059158  …                0.102389
18  0.033473  0.025105  0.092050  0.079498  …               -1.000000
19  0.055556  0.039683  0.079365  0.095238  …               -1.000000
20  0.067308  0.019231  0.067308  0.076923  …               -1.000000
21  0.057143  0.019048  0.066667  0.076190  …               -1.000000
22  0.051676  0.018156  0.064246  0.071229  …               -1.000000
23  0.035714  0.020408  0.071429  0.076531  …               -1.000000
24  0.032787  0.016393  0.049180  0.073770  …               -1.000000
25  0.038014  0.038014  0.071373  0.074476  …                0.070597
26  0.033097  0.027187  0.075650  0.065012  …                0.106383
27  0.049830  0.022650  0.056625  0.092865  …                0.098528
28  0.045254  0.029801  0.070640  0.071744  …                0.097130
29  0.075802  0.032070  0.075802  0.067055  …               -1.000000

    dist_kinase2_glpl_norm  dist_glpl_mhd_norm  has_ploop  has_kinase2  \
0                -1.000000                   1          0            0
1                -1.000000                   0          0            1
2                 0.108136                   1          1            1
3                -1.000000                   0          0            0
4                -1.000000                   1          0            1
5                 0.128350                   0          0            1
6                 0.288288                   1          0            1
7                 0.202614                   0          0            1
8                 0.096190                   1          1            1
9                -1.000000                   0          0            1
10               -1.000000                   0          0            1
11                0.295047                   1          1            1
12                0.312500                   0          0            1
13                0.235981                   1          0            1
14               -1.000000                   1          1            1
15                0.093679                   1          1            1
16               -0.346154                   1          1            1
17                0.145620                   1          1            1
18               -1.000000                   1          0            0
19               -1.000000                   0          0            0
20               -1.000000                   0          0            1
21               -1.000000                   1          0            0
22                0.139665                   1          0            1
23               -1.000000                   0          1            0
24               -1.000000                   0          0            0
25               -0.452289                   1          1            1
26                0.102837                   1          1            1
27                0.412231                   1          1            1
28                0.143488                   1          1            1
29               -1.000000                   0          0            0

    has_glpl  has_mhd  target              id   cluster
```

```
0          0        1       1  NP_001234459.3  POS_1502
1          0        1       1  NP_001308492.1  POS_1005
2          1        4       1    CAQ5402917.1   POS_548
3          0        0       1    CAQ5403257.1  POS_1404
4          0        2       1    CAQ5404663.1   POS_614
5          1        2       1    CAQ5406153.1  POS_1000
6          1        3       1    CAQ5404100.1   POS_504
7          1        2       1    CAQ5404098.1   POS_427
8          1        4       1    CAQ5403806.1   POS_449
9          0        1       1    CAQ5409432.1   POS_681
10         0        1       1    CAQ5408022.1   POS_356
11         1        4       1    CAQ5407207.1   POS_585
12         1        2       1    CAQ5411306.1  POS_1456
13         1        3       1    CAQ5411102.1  POS_1278
14         0        3       1    CAQ5410995.1  POS_1472
15         1        4       1    CAQ5413609.1   POS_723
16         1        4       1    CAQ5413612.1   POS_663
17         1        4       1    CAQ5410853.1   POS_740
18         0        1       1    CAQ5410852.1  POS_1491
19         0        0       1    CAQ5413789.1  POS_1696
20         0        1       1    CAQ5410659.1  POS_1709
21         0        1       1    CAQ5410656.1  POS_1708
22         1        3       1    CAQ5410654.1   POS_994
23         1        2       1    CAQ5410652.1  POS_1573
24         1        1       1    CAQ5410649.1  POS_1699
25         1        4       1    CAQ5410600.1   POS_211
26         1        4       1    CAQ5413906.1   POS_810
27         1        4       1    CAQ5410544.1   POS_729
28         1        4       1    CAQ5413907.1   POS_675
29         0        0       1    CAQ5413931.1  POS_1378

[30 rows x 448 columns]
```

[7]:
```
inicio_otimizacao = datetime.now()
print("\n\n\nInciando Otimização dos modelos: ", inicio_otimizacao.strftime("%H:
  ↪%M:%S"))

scores = []
groups_train = train_df["cluster"]

scoring = {
    'mcc': make_scorer(matthews_corrcoef),
    'recall':  make_scorer(recall_score),
    'precision':  make_scorer(precision_score),
    'f1':  make_scorer(f1_score),
    'roc_auc': 'roc_auc',
}
```

```
cv_t = GroupKFold(n_splits=5)

# Otimização XGB
param_dist_xgb = {
    'n_estimators':[500,800,1200,1500],
    'max_depth':[3,4,5,6,7,10],
    'learning_rate':[0.01,0.03,0.05,0.1],
    'subsample':[0.7,0.8,0.9],
    'colsample_bytree':[0.7,0.8,0.9],
    'min_child_weight':[1,3,5],
    'reg_alpha':[0,0.1,0.5,0.7],
    'reg_lambda':[1,2,5]
}
print('\n\nOtimizando XGBoost...')
search_xgb = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_dist_xgb,
    n_iter=50,
    scoring=scoring,
    refit='f1',
    cv=cv_t,
    random_state=42,
    verbose=2,
    n_jobs=-1
)

search_xgb.fit(
    X_train,
    y_train,
    groups=groups_train
)
print("\nMelhores parâmetros para o XGB:")
print(search_xgb.best_params_)

print("\nMelhor Recall:")
print(search_xgb.best_score_)

best_params = search_xgb.best_params_
xgb = search_xgb.best_estimator_
```

```
Inciando Otimização dos modelos:  14:20:13
```

```
Otimizando XGBoost…
Fitting 5 folds for each of 50 candidates, totalling 250 fits

Melhores parâmetros para o XGB:
{'subsample': 0.8, 'reg_lambda': 1, 'reg_alpha': 0.5, 'n_estimators': 1500,
'min_child_weight': 1, 'max_depth': 3, 'learning_rate': 0.05,
'colsample_bytree': 0.7}

Melhor Recall:
0.8920809070696771
```

[8]:
```python
# otimização RF

param_dist_rf = {
    'n_estimators':[500,1000,1200,1500],
    'max_depth':[8,12,16, None],
    'min_samples_split':[2,4,5,8],
    'min_samples_leaf':[1,2,4],
    'max_features':['sqrt','log2',0,3],
    'class_weight':['balanced','balanced_subsample']
}

print('\n\nOtimizando RF...')
search_rf = RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_dist_rf,
    n_iter=40,
    scoring=scoring,
    refit='f1',
    cv=cv_t,
    random_state=42,
    verbose=2,
    n_jobs=-1
)

search_rf.fit(
    X_train,
    y_train,
    groups=groups_train
)
print("\nMelhores parâmetros para o RF:")
print(search_rf.best_params_)

print("\nMelhor Recall:")
print(search_rf.best_score_)

rf = search_rf.best_estimator_
```

```
rf.set_params(n_jobs=-1)
```

```
Otimizando RF…
Fitting 5 folds for each of 40 candidates, totalling 200 fits

Melhores parâmetros para o RF:
{'n_estimators': 1000, 'min_samples_split': 8, 'min_samples_leaf': 1,
'max_features': 'sqrt', 'max_depth': None, 'class_weight': 'balanced_subsample'}

Melhor Recall:
0.8609705821511874
```

[8]:
```
RandomForestClassifier(class_weight='balanced_subsample', min_samples_split=8,
                       n_estimators=1000, n_jobs=-1, random_state=42)
```

[9]:
```
print('\n\nOtimizando SVM...')
param_dist_svm = {
    'C':[0.5,1,2,5,10,20,50],
    'gamma':[
        'scale',
        0.01,
        0.005,
        0.001,
        0.0005
    ],
    'kernel':['rbf']
}
search_svm = RandomizedSearchCV(
    estimator=svm,
    param_distributions=param_dist_svm,
    n_iter=25,
    scoring=scoring,
    refit='f1',
    cv=cv_t,
    random_state=42,
    verbose=2,
    n_jobs=-1
)

search_svm.fit(
    X_train,
    y_train,
    groups=groups_train
)
print("\nMelhores parâmetros para o SVM:")
```

```
print(search_svm.best_params_)

print("\nMelhor Recall:")
print(search_svm.best_score_)

svm = search_svm.best_estimator_

fim = datetime.now()
print("\n\nFinalizado otimização dos modelos em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio_otimizacao)
# Fim otimização
```

```
Otimizando SVM…
Fitting 5 folds for each of 25 candidates, totalling 125 fits

Melhores parâmetros para o SVM:
{'kernel': 'rbf', 'gamma': 0.001, 'C': 2}

Melhor Recall:
0.8899564834649676


Finalizado otimização dos modelos em: 14:36:47

Tempo decorrido: 0:16:33.784780
```

[10]:
```
print("\n\n Inciando validação cruzada");
scores_rf = validacao_cruzada('RF',rf,X_train, y_train, groups_train)
scores_xgb = validacao_cruzada('XGB',xgb, X_train, y_train, groups_train)
scores_svm = validacao_cruzada('SVM',svm, X_train, y_train, groups_train)

print("Cross-Validation:RF")
for k,v in scores_rf.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

print("Cross-Validation XGB")
for k,v in scores_xgb.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

print("Cross-Validation SVM")
for k,v in scores_svm.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")
```

```
resultados_cv = pd.DataFrame({
    'Modelo': ['RF','XGB','SVM'],
    'Accuracy': [
        scores_rf['test_accuracy'].mean(),
        scores_xgb['test_accuracy'].mean(),
        scores_svm['test_accuracy'].mean(),
    ],
    'Recall': [
        scores_rf['test_recall'].mean(),
        scores_xgb['test_recall'].mean(),
        scores_svm['test_recall'].mean(),
    ],
    'Precision': [
        scores_rf['test_precision'].mean(),
        scores_xgb['test_precision'].mean(),
        scores_svm['test_precision'].mean(),
    ],
    'F1': [
        scores_rf['test_f1'].mean(),
        scores_xgb['test_f1'].mean(),
        scores_svm['test_f1'].mean(),
    ],
    'ROC_AUC': [
        scores_rf['test_roc_auc'].mean(),
        scores_xgb['test_roc_auc'].mean(),
        scores_svm['test_roc_auc'].mean(),
    ],
    'MCC': [
        scores_rf['test_mcc'].mean(),
        scores_xgb['test_mcc'].mean(),
        scores_svm['test_mcc'].mean(),
    ]
})
print(resultados_cv)
resultados_cv.to_csv(
        f"{OUTPUT_DIR}/metricas_cv.csv",
        index=False
    )
dados = []

for score in scores_rf['test_roc_auc']:
    dados.append(['RF',score])
for score in scores_xgb['test_roc_auc']:
    dados.append(['XGB',score])
for score in scores_svm['test_roc_auc']:
```

```
    dados.append(['SVM',score])

grafico = pd.DataFrame(dados, columns=['Modelo','ROC_AUC'])
grafico.boxplot(by='Modelo')
plt.ylabel("ROC-AUC")
# plt.title("10-Fold Cross Validation")
plt.suptitle("")
plt.tight_layout()
# plt.show()
plt.savefig(
    f"{OUTPUT_DIR}/kfold_cross_validation_roc.png",
    bbox_inches="tight"
)
plt.close()
```

```
 Inciando validação cruzada


==========================



Validação:  RF


Recall:
[0.76811594 0.75362319 0.76811594 0.80434783 0.78985507 0.78985507
 0.76086957 0.75912409 0.75912409 0.73913043]


MCC:
[0.76959638 0.75727179 0.76959638 0.79231045 0.78827875 0.80580935
 0.76342156 0.78103164 0.76308624 0.71826047]


ACC:
[0.875      0.86764706 0.875      0.88970588 0.88602941 0.89338235
 0.87132353 0.87867647 0.87132353 0.84926471]


ROC:
[0.94002812 0.91780229 0.91910015 0.96355181 0.9362427  0.95241185
 0.94603072 0.94722898 0.94225466 0.93310621]
```

```
F1:
[0.86178862 0.85245902 0.86178862 0.88095238 0.87550201 0.88259109
 0.85714286 0.86307054 0.85596708 0.83265306]


==========================


Finalizada a validação cruzada para: RF


==========================




Validação:  XGB


Recall:
[0.84782609 0.82608696 0.83333333 0.87681159 0.87681159 0.86956522
 0.84057971 0.83941606 0.83941606 0.82608696]


MCC:
[0.79062974 0.76233146 0.75320142 0.81810613 0.81810613 0.8190334
 0.76794882 0.82442449 0.75215565 0.79492301]


ACC:
[0.89338235 0.87867647 0.875      0.90808824 0.90808824 0.90808824
 0.88235294 0.90808824 0.875      0.89338235]


ROC:
[0.94094744 0.94770712 0.92142548 0.95944192 0.94332684 0.96263249
 0.94067705 0.95139227 0.94695864 0.93424183]


F1:
[0.88973384 0.87356322 0.87121212 0.90636704 0.90636704 0.90566038
 0.87878788 0.90196078 0.87121212 0.88715953]


==========================


Finalizada a validação cruzada para: XGB
```

```
=========================



Validação:  SVM


Recall:
[0.84057971 0.83333333 0.80434783 0.85507246 0.86231884 0.87681159
 0.86956522 0.81751825 0.83941606 0.8115942 ]


MCC:
[0.8081097  0.80149986 0.78381394 0.80538556 0.78865215 0.83377319
 0.82693839 0.81339466 0.78372441 0.74891868]


ACC:
[0.90073529 0.89705882 0.88602941 0.90073529 0.89338235 0.91544118
 0.91176471 0.90073529 0.88970588 0.87132353]


ROC:
[0.95771144 0.93948735 0.93856803 0.96192948 0.9429483  0.95500757
 0.95711659 0.93863206 0.95560962 0.94110967]


F1:
[0.8957529  0.89147287 0.87747036 0.8973384  0.89138577 0.91320755
 0.90909091 0.89243028 0.88461538 0.86486486]


=========================


Finalizada a validação cruzada para: SVM
Cross-Validation:RF
test_recall: 0.7692 ± 0.0187
test_roc_auc: 0.9398 ± 0.0134
test_mcc: 0.7709 ± 0.0227
test_accuracy: 0.8757 ± 0.0119
test_precision: 0.9815 ± 0.0124
test_f1: 0.8624 ± 0.0141
Cross-Validation XGB
test_recall: 0.8476 ± 0.0187
```

```
test_roc_auc: 0.9449 ± 0.0113
test_mcc: 0.7901 ± 0.0277
test_accuracy: 0.8930 ± 0.0137
test_precision: 0.9355 ± 0.0196
test_f1: 0.8892 ± 0.0142
Cross-Validation SVM
test_recall: 0.8411 ± 0.0236
test_roc_auc: 0.9488 ± 0.0089
test_mcc: 0.7994 ± 0.0232
test_accuracy: 0.8967 ± 0.0120
test_precision: 0.9497 ± 0.0175
test_f1: 0.8918 ± 0.0134
  Modelo  Accuracy    Recall  Precision        F1   ROC_AUC       MCC
0     RF  0.875735  0.769216   0.981547  0.862392  0.939776  0.770866
1    XGB  0.893015  0.847593   0.935508  0.889202  0.944875  0.790086
2    SVM  0.896691  0.841056   0.949688  0.891763  0.948812  0.799421
```

[ ]:
```python
################################################
# Treinamento
inicio = datetime.now()
print("\n\n\nInciando treinamento RF: ", inicio.strftime("%H:%M:%S"))
results = []
r1, rf_model = avaliar_modelo(
    "Random Forest",
    rf,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r1)
fim = datetime.now()
print("\n\nFinalizado treinamento RF em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_e_salvar_shap("Random Forest", rf_model, X_test, columns,OUTPUT_DIR )

inicio = datetime.now()
print("\n\n\nInciando treinamento XGBoost: ", inicio.strftime("%H:%M:%S"))
r2, xgb_model = avaliar_modelo(
    "XGBoost",
    xgb,
    X_train,
    y_train,
    X_test,
    y_test
)
```

```
results.append(r2)
fim = datetime.now()
print("\n\nFinalizado em XGBoost:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_e_salvar_shap("XGBoost", xgb_model, X_test, columns,OUTPUT_DIR )

inicio = datetime.now()
print("\n\n\nInciando treinamento SVM: ", inicio.strftime("%H:%M:%S"))
r3, svm_model = avaliar_modelo(
    "SVM",
    svm,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r3)
fim = datetime.now()
print("\n\nFinalizado em SVM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_shap_svm("SVM", svm_model,columns, X_train, X_test, y_test, OUTPUT_DIR)

fim = datetime.now()
print("\n\nFinalizado fase de treinamento em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio_treinamento)

print("\nFim criação/treino dos modelos\n")
```

```
Inciando treinamento RF:  14:37:33

============================================================
Random Forest
============================================================
              precision    recall  f1-score   support

           0       0.85      0.98      0.91       358
           1       0.98      0.80      0.88       322

    accuracy                           0.90       680
   macro avg       0.91      0.89      0.90       680
weighted avg       0.91      0.90      0.90       680

ROC-AUC          : 0.9542749574933203
```

```
PR-AUC           : 0.9618177020542162
MCC              : 0.8037186718808346
F1               : 0.8805460750853242
Precision        : 0.9772727272727273
Recall           : 0.8012422360248447
Balanced Accuracy: 0.8922412297442659
Standard Accuracy: 0.8970588235294118


Finalizado treinamento RF em: 14:37:36

Tempo decorrido: 0:00:02.331482

[SHAP] Inicializando análise de interpretabilidade para: Random Forest…
[SHAP] Gráficos salvos com sucesso em 'pipelineAG/'!



Inciando treinamento XGBoost:  14:38:09

============================================================
XGBoost
============================================================
              precision    recall  f1-score   support

           0       0.90      0.96      0.92       358
           1       0.95      0.88      0.91       322

    accuracy                           0.92       680
   macro avg       0.92      0.92      0.92       680
weighted avg       0.92      0.92      0.92       680

ROC-AUC          : 0.963166660883445
PR-AUC           : 0.9644334860365208
MCC              : 0.8363236524095334
F1               : 0.9096774193548387
Precision        : 0.9463087248322147
Recall           : 0.8757763975155279
Balanced Accuracy: 0.9155418300426802
Standard Accuracy: 0.9176470588235294


Finalizado em XGBoost: 14:38:16

Tempo decorrido: 0:00:07.307192

[SHAP] Inicializando análise de interpretabilidade para: XGBoost…
[SHAP] Gráficos salvos com sucesso em 'pipelineAG/'!
```

```
Inciando treinamento SVM:  14:38:17

============================================================
SVM
============================================================
              precision    recall  f1-score   support

           0       0.90      0.94      0.92       358
           1       0.93      0.89      0.91       322

    accuracy                           0.92       680
   macro avg       0.92      0.91      0.92       680
weighted avg       0.92      0.92      0.92       680

ROC-AUC          : 0.9570335542524029
PR-AUC           : 0.9615403671046813
MCC              : 0.8324869972206768
F1               : 0.9090909090909091
Precision        : 0.9344262295081968
Recall           : 0.8850931677018633
Balanced Accuracy: 0.9146136229570769
Standard Accuracy: 0.9161764705882353


Finalizado em SVM: 14:38:18

Tempo decorrido: 0:00:01.671509

[SVM] Calculando a importância por permutação dos atributos…

Top 20 features mais importantes para o SVM:
      feature  importance_mean
110  DIPEP_FL         0.004412
326  DIPEP_SG         0.003824
38   DIPEP_AV         0.003529
143  DIPEP_HD         0.003529
204  DIPEP_LE         0.003529
187  DIPEP_KH         0.003529
346  DIPEP_TG         0.003235
36   DIPEP_AS         0.003235
175  DIPEP_IR         0.003235
209  DIPEP_LK         0.003235
270  DIPEP_PL         0.003235
124  DIPEP_GE         0.002941
144  DIPEP_HE         0.002941
```

```
215  DIPEP_LR         0.002941
203  DIPEP_LD         0.002941
9       AAC_K         0.002941
13      AAC_P         0.002941
315  DIPEP_RR         0.002941
126  DIPEP_GG         0.002941
155  DIPEP_HR         0.002941

Criando explainer shap…


PermutationExplainer explainer:   9%|          | 18/200 [04:46<50:20, 16.60s/it]
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.4min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  57.3s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  39.4s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  36.2s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  39.2s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  51.7s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  26.6s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  42.3s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  39.5s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.1min
```

```
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  50.9s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  35.2s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=   8.4s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   5.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  12.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  11.1s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  19.8s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  17.8s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  17.1s
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  18.4s
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  21.0s
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  25.9s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time=  42.8s
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  22.2s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  21.3s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  44.1s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.4min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  56.8s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  45.7s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  50.8s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  33.1s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
```

```
total time=  22.9s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  49.2s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  35.9s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   5.0s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   5.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  13.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   6.2s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   7.8s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  19.1s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  17.5s
[CV] END …C=5, gamma=scale, kernel=rbf; total time=  25.6s
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  19.6s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  19.5s
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  19.6s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time=  39.1s
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  23.2s
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=  11.5s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  33.2s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  43.1s
```

```
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  45.9s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.0min
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  51.8s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.6min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  52.5s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  30.0s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time=  53.7s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  22.6s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  42.5s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  21.4s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   4.2s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   2.8s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.6s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
```

```
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  24.6s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  23.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  11.1s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   9.4s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   6.4s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time=  43.0s
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  22.0s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  22.0s
[CV] END …C=20, gamma=0.005, kernel=rbf; total time=  37.5s
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time=  33.6s
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  11.3s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  20.6s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  43.4s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.7min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.3min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  16.7s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 2.9min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  35.2s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  14.1s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  28.1s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  41.5s
```

```
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  36.2s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   5.3s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   5.3s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  13.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   5.5s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   7.7s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   9.4s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   6.4s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time=  42.6s
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  22.1s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  21.5s
[CV] END …C=5, gamma=0.005, kernel=rbf; total time=  37.3s
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time=  26.7s
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  21.6s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  21.5s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  45.4s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.5min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.3min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
```

```
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  17.5s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 2.8min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  34.5s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  14.5s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.6min
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   4.2s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   2.7s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.4s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   7.8s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   7.6s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  21.4s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  21.4s
```

```
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  23.8s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time=  43.3s
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  21.1s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  20.2s
[CV] END …C=5, gamma=0.005, kernel=rbf; total time=  39.9s
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time=  27.3s
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  21.2s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  21.0s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  45.9s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.7min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.3min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  17.9s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  49.3s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  33.6s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  23.4s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  39.8s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  28.4s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  16.0s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  51.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
```

```
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   5.2s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   5.3s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  21.1s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  21.4s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   6.3s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   5.5s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   6.5s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  24.2s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time=  41.3s
[CV] END …C=2, gamma=0.005, kernel=rbf; total time=  41.3s
[CV] END …C=5, gamma=0.005, kernel=rbf; total time=  39.2s
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time=  30.7s
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  21.0s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.7min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.3min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  17.4s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  49.2s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  34.6s
```

```
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  24.2s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  40.0s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  21.9s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  42.0s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  27.9s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  24.6s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  21.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  21.3s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  23.2s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  21.8s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  15.3s
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  16.9s
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  18.9s
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  24.4s
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  25.6s
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time=  32.2s
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  17.8s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  21.3s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  46.6s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.3min
```

```
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  40.5s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  37.1s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  41.4s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  50.9s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  26.0s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  42.9s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  40.1s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  50.4s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  27.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   5.1s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   5.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  11.3s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  21.2s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   5.1s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   5.0s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  10.1s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   6.4s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
```

```
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  24.8s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time=  39.4s
[CV] END …C=2, gamma=0.005, kernel=rbf; total time=  40.6s
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  16.4s
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  21.2s
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time=  38.6s
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  14.1s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  32.1s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  43.9s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  46.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  57.5s
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  51.3s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.6min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  52.1s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  29.9s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time=  53.4s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
```

```
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  28.6s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  15.4s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  45.1s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   4.2s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   2.7s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.5s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   7.7s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  24.1s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   9.4s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   6.3s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  11.3s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  19.7s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  18.5s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  17.1s
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  17.6s
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  22.6s
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  21.5s
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  27.5s
[CV] END …C=50, gamma=0.005, kernel=rbf; total time=  39.2s
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=  12.1s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  21.7s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  47.7s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
```

```
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.4min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  58.1s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  56.5s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  44.0s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 2.7min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  33.8s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  14.3s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.7min
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  24.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  20.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  20.6s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  23.7s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  15.6s
[CV] END …C=5, gamma=scale, kernel=rbf; total time=  26.9s
[CV] END …C=2, gamma=0.005, kernel=rbf; total time=  40.6s
[CV] END …C=5, gamma=0.005, kernel=rbf; total time=  34.0s
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time=  40.2s
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  11.6s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.4min
```

```
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  57.2s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  54.6s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  44.8s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 2.8min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  34.2s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  15.3s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.7min
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  25.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  21.3s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  21.7s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  22.8s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  21.8s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  14.2s
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  14.6s
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  21.5s
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  21.6s
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  25.3s
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time=  42.2s
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=  14.5s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  20.8s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  46.0s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.7min
```

```
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.2min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  17.3s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 2.8min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  33.9s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  14.1s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.6min
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  24.1s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   9.3s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   9.4s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   5.1s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   5.0s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=   9.9s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   6.2s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  23.1s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  17.4s
[CV] END …C=5, gamma=scale, kernel=rbf; total time=  32.4s
[CV] END …C=5, gamma=0.01, kernel=rbf; total time=  37.9s
[CV] END …C=20, gamma=0.005, kernel=rbf; total time=  37.1s
[CV] END …C=50, gamma=0.005, kernel=rbf; total time=  40.4s
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=  10.7s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  33.2s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  42.0s
```

```
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  46.3s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.0min
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  50.3s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.6min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  50.8s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  27.8s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  43.8s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  37.4s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  41.2s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  27.3s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  16.5s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  51.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   5.1s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   5.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  11.3s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  11.3s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
```

```
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=   8.6s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=   8.4s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  10.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   5.4s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   7.9s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   9.4s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   6.4s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time=  39.9s
[CV] END …C=2, gamma=0.005, kernel=rbf; total time=  37.3s
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  21.3s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time=  41.4s
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  20.4s
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=  15.5s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  20.9s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  44.7s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.6min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  55.1s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  41.0s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  37.1s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  41.1s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  51.8s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  27.5s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
```

```
subsample=0.7; total time=  45.0s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  38.6s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  41.9s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  27.8s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  15.9s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  50.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   5.0s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   5.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  11.4s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  11.3s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=   8.5s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=   8.4s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  10.2s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   5.4s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   7.9s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   9.4s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   6.7s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time=  44.2s
[CV] END …C=5, gamma=0.01, kernel=rbf; total time=  40.1s
[CV] END …C=5, gamma=0.005, kernel=rbf; total time=  37.0s
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time=  36.9s
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  13.7s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  21.3s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
```

```
subsample=0.7; total time=  41.3s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.2min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.4min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  40.9s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  39.1s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  42.2s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  53.5s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  30.1s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time=  55.6s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  22.4s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  16.1s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  49.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   5.2s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   5.3s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  21.1s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  21.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   6.2s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
```

```
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   5.7s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   8.0s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   9.5s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   6.3s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time=  43.7s
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  20.3s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  18.3s
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  22.5s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time=  38.8s
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  22.2s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.0min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.2min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.4min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  58.1s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  44.8s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  48.1s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  33.9s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  22.6s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  41.0s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.5min
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   4.2s
```

```
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   2.7s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.5s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   7.7s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   7.6s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  21.1s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  21.6s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  24.1s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time=  40.9s
[CV] END …C=2, gamma=0.005, kernel=rbf; total time=  28.8s
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  20.2s
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  22.9s
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time=  40.4s
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  20.1s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  21.9s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  46.6s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.3min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  39.3s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  38.7s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  42.9s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  51.0s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  28.4s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
```

```
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  41.9s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  37.8s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  49.9s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  35.8s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   5.0s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   5.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  13.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   6.1s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   6.3s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  23.9s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time=  44.1s
[CV] END …C=5, gamma=0.01, kernel=rbf; total time=  43.5s
[CV] END …C=20, gamma=0.005, kernel=rbf; total time=  39.1s
[CV] END …C=50, gamma=0.005, kernel=rbf; total time=  40.8s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  32.9s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  44.2s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  47.0s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  59.0s
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  51.5s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
```

```
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.6min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  52.4s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  31.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time=  52.9s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  22.8s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  42.6s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  21.7s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   4.4s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   2.8s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.5s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  23.8s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  24.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  11.0s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  19.8s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  17.1s
[CV] END …C=5, gamma=scale, kernel=rbf; total time=  31.8s
[CV] END …C=5, gamma=0.01, kernel=rbf; total time=  37.9s
[CV] END …C=20, gamma=0.005, kernel=rbf; total time=  40.8s
[CV] END …C=50, gamma=0.005, kernel=rbf; total time=  38.4s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.2min
```

```
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.4min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  56.3s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  56.6s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  44.4s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  49.3s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  35.1s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  21.8s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  48.9s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  36.1s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   5.3s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   5.5s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   5.4s
```

```
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  10.0s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   6.3s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  22.6s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  22.6s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  16.3s
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  12.8s
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  19.6s
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  26.7s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time=  40.2s
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  19.5s
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  11.4s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  32.7s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  43.6s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  47.6s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  57.9s
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  50.8s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.6min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  53.1s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  30.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time=  54.4s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
```

```
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  22.3s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  40.7s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  27.5s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  24.1s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   9.2s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   9.3s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   5.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  13.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  11.1s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  19.6s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  15.5s
[CV] END …C=5, gamma=scale, kernel=rbf; total time=  28.5s
[CV] END …C=5, gamma=0.01, kernel=rbf; total time=  42.7s
[CV] END …C=20, gamma=0.005, kernel=rbf; total time=  40.5s
[CV] END …C=50, gamma=0.005, kernel=rbf; total time=  39.7s

PermutationExplainer explainer:  10%|         | 21/200 [05:38<51:01, 17.10s/it]
```

[ ]:
```
'''
Avaliação e Validação dos modelos
'''
print("\n\nInciando Validação dos modelos\n")
# verificar importância dos atributos/feature para o Random Forest
```

```
importancia = pd.DataFrame({"feature":columns,"importance":rf_model.
 ↪feature_importances_})
importancia = importancia.sort_values("importance",ascending = False)
print("\nTop 20 features: Random Forest")
print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh( top["feature"][::-1], top["importance"][::-1] )
plt.xlabel("Importância")
plt.ylabel("Feature")
# plt.title("Top 20 Features - Random Forest")
plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/rf_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()

# verificar importância dos atributos/feature para o XGBoost
importancia = pd.DataFrame({"feature":columns, "importance":xgb_model.
 ↪feature_importances_})
importancia = importancia.sort_values("importance", ascending = False)
print("\nTop 20 features: XGBoost")
print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh(top["feature"][::-1], top["importance"][::-1])
plt.xlabel("Importância")
plt.ylabel("Feature")
# plt.title("Top 20 Features - XGBoost")
plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/xgb_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()

print("\nCOMPARAÇÃO MODELOS")
print(pd.DataFrame(results))

results_df = pd.DataFrame(results)
results_df.to_csv(
        f"{OUTPUT_DIR}/metricas_comparacao.csv",
        index=False
    )
```

```
metrics = [
    "ROC_AUC",
    "F1",
    "Precision",
    "Recall",
    "Balanced_Accuracy"
]

for metric in metrics:
    plt.figure(figsize=(8,5))
    plt.bar(
        results_df["Modelo"],
        results_df[metric]
    )

    plt.ylim(0, 1)
    plt.ylabel(metric)
    # plt.title(f"Comparação dos Modelos - {metric}")
    for i, v in enumerate(results_df[metric]):
        plt.text(i, v + 0.01, f"{v:.3f}", ha='center')

    plt.savefig(
        f"{OUTPUT_DIR}/comparacao_{metric}.png",
        bbox_inches="tight"
    )
    plt.close()

modelos = {
    "Random Forest":rf_model,
    "XGBoost":xgb_model,
    "SVM":svm_model
    }
plt.figure(figsize=(7,7))
for nome, model in modelos.items():
    probs = model.predict_proba(X_test)[:,1]
    fpr, tpr, _ = roc_curve(y_test, probs)
    roc_auc = auc(fpr, tpr)
    plt.plot(
        fpr,
        tpr,
        label=f"{nome} AUC={roc_auc:.3f}"
    )
plt.plot([0,1],[0,1],'--')
plt.xlabel("FPR")
plt.ylabel("TPR")
# plt.title("Comparação ROC")
```

```
plt.legend()
plt.savefig(f"{OUTPUT_DIR}/roc_comparativa.png")
plt.close()

print("Fim avaliação / validação modelos ")
```

[ ]:
```
'''
Funções para rodar a aplicação
'''

df_validacao = []

def validar_motifs(df,planta,modelo):
    print("\nVALIDAÇÃO BIOLÓGICA\n")
    counts = {}
    for motif_name, pattern in motifs.items():
        counts[motif_name] = 0
        for seq in df["protein_seq"]:
            if re.search(pattern, seq):
                counts[motif_name] += 1
    for k, v in counts.items():
        print(k, ":", v)

    counts['planta'] = planta
    counts['modelo'] = modelo

    df_validacao.append(counts)
    df_counts = pd.DataFrame([counts])
    df_counts.to_csv(
        f"{OUTPUT_DIR}/validacao_biologica_{modelo}_{planta}.csv",
        index=False
    )



def rodar_transdecoder(fasta_nt):
    cmd1 = [
        "TransDecoder.LongOrfs",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
    ]
    with open(fasta_nt+"_output1.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd1,
                       stdout=log_file,
                       stderr=subprocess.STDOUT,
```

```
                        text=True)

    cmd2 = [
        "TransDecoder.Predict",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
    ]
    with open(fasta_nt+"_output2.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd2,
                        stdout=log_file,
                        stderr=subprocess.STDOUT,
                        text=True)
    pep_file = fasta_nt + ".transdecoder.pep"
    return pep_file


def predizer_transcritos(fasta_nt,model,nmodel, output, planta):
    probabilidade = 0.85
    candidatos = []
    pep_file = rodar_transdecoder(fasta_nt)
    for record in SeqIO.parse(fasta_nt+".transdecoder.pep", "fasta"):
        protein = str(record.seq)
        feats = extrair_features_proteina(protein)
        if feats is None:
            continue
        X = scaler.transform([feats])
        prob = model.predict_proba(X)[0][1]
        candidatos.append({
            "id": record.id,
            "descricao": record.description,
            "protein_length": len(protein),
            "probabilidade_resistencia": prob,
            "protein_seq": protein
        })

    candidatos_df = pd.DataFrame(candidatos)
    candidatos_df = candidatos_df.sort_values(
        "probabilidade_resistencia",
        ascending=False
    )

    # top = candidatos_df.head(40)
    top = candidatos_df[candidatos_df["probabilidade_resistencia"] >=␣
↪probabilidade]
    print("\nTOP CANDIDATOS com probabilidade de >= ")
```

```
    print(probabilidade)
    print(fasta_nt+", MODELO: "+nmodel)
    print(top[[
        "id",
        "descricao",
        "protein_length",
        "probabilidade_resistencia"
    ]])

    validar_motifs(top,planta,nmodel)

    candidatos_df.to_csv(
        f"{OUTPUT_DIR}/{output}",
        index=False
    )
```

[ ]:
```
'''
Aplicação - Modelo XGBoost
'''
print("\n\n=================Inciando Aplicação: Predição de␣
 ↪Genes===============\n\n")
print("\n\n Predição Modelo XGBoost")
inicio = datetime.now()
print("\n\n\nInciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))

print("\nPredição para ARABIDOPSIS\n")
predizer_transcritos(transcriptoma_arabidopsis_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_arabidopsis_xgboost.
 ↪csv",planta="Arabidopsis thaliana")

fim = datetime.now()
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

[ ]:
```
print("\nPredição para MELONGENA\n")
inicio = datetime.now()
print("\n\n\nInciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_melongena_xgboost.csv",planta="Solanum␣
 ↪melongena")
fim = datetime.now()
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

[ ]:
```
print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\n\nInciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
```

```
predizer_transcritos(transcriptoma_tuberosum_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_tuberosum_xgboost.csv",planta="Solanum␣
 ↪tuberosum")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

[ ]:
```
print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_physalis_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_physalis_xgboost.csv",planta="Physalis␣
 ↪peruviana")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_xgboost.csv")
```

[ ]:
```
print(df_validacao)
```

[ ]:
```
'''
Aplicação - Modelo Random Forest
'''
print("\n\n Predição Modelo RF")
print("\nPredição para ARABIDOPSIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_arabidopsis_fasta,␣
 ↪rf_model,"RF",output="predicoes_arabidopsis_rf.csv",planta="Arabidopsis␣
 ↪thaliana")
fim = datetime.now()
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

[ ]:
```
print("\nPredição para MELONGENA\n")
inicio = datetime.now()
print("\n\n\nInciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,␣
 ↪rf_model,'RF',output="predicoes_melongena_rf.csv",planta="Solanum melongena")
fim = datetime.now()
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

[ ]:
```
print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\n\nInciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
```

```
predizer_transcritos(transcriptoma_tuberosum_fasta,␣
  ↪rf_model,"RF",output="predicoes_tuberosum_rf.csv",planta="Solanum tuberosum")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

[ ]:
```
print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_physalis_fasta,␣
  ↪rf_model,"RF",output="predicoes_physalis_rf.csv",planta="Physalis peruviana")
fim = datetime.now()
print("\n\nFinalizado predição PHYSALIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_rf.csv")
```

[ ]:
```
'''
Aplicação - Modelo SVM
'''
print("\n\n Predição Modelo SVM")
print("\nPredição para ARABIDOPSIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_arabidopsis_fasta,␣
  ↪svm_model,"SVM",output="predicoes_arabidopsis_svm.csv",planta="Arabidopsis␣
  ↪thaliana")
fim = datetime.now()
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

[ ]:
```
print("\nPredição para MELONGENA\n")
inicio = datetime.now()
print("\n\n\nInciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,␣
  ↪svm_model,'SVM',output="predicoes_melongena_svm.csv",planta="Solanum␣
  ↪melongena")
fim = datetime.now()
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

[ ]:
```
print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\n\nInciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
```

```
predizer_transcritos(transcriptoma_tuberosum_fasta,␣
 ↪svm_model,"SVM",output="predicoes_tuberosum_svm.csv",planta="Solanum␣
 ↪tuberosum")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

[ ]:
```
print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_physalis_fasta,␣
 ↪svm_model,"SVM",output="predicoes_physalis_svm.csv",planta="Physalis␣
 ↪peruviana")
fim = datetime.now()
print("\n\nFinalizado predição PHYSALIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_svm.csv")
```

[ ]:
```
print(df_validacao)
```

[ ]:
```
df_copia = df_validacao.copy()
df_inicial = pd.DataFrame(df_validacao)

df_validacao = df_inicial.melt(
    id_vars=["planta", "modelo"],
    var_name="motivo",
    value_name="contagem"
)
print(df_validacao.head())

df_validacao.to_csv(f"{OUTPUT_DIR}/df_validacao.csv", index=False)
```

[ ]:
```
#Scatterplot
plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df_validacao,
    x="motivo",
    y="contagem",
    hue="planta",
    style="modelo",
    s=200
)
# plt.title( "Validação biológica dos motivos conservados")
plt.ylabel("Número de ocorrências")
plt.xlabel("Motivo")
```

```
plt.legend(
    bbox_to_anchor=(1.05,1),
    loc="upper left"
)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/Validacao_biologica_dos_motivos_conservados.png")
plt.close()
# plt.show()

#Bubble Chart
plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df_validacao,
    x="motivo",
    y="modelo",
    hue="planta",
    size="contagem",
    sizes=(50,600)
)
# plt.title("Distribuição dos motivos por modelo e espécie")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/distribuicao_dos_motivos_por_modelo_especie.png")
plt.close()
# plt.show()


#Facetas por planta
g = sns.catplot(
    data=df_validacao,
    x="motivo",
    y="contagem",
    hue="modelo",
    col="planta",
    kind="bar",
    height=5,
    aspect=1.2
)
g.set_xticklabels(rotation=45)
plt.savefig(f"{OUTPUT_DIR}/facetas_por_planta.png")
plt.close()
# plt.show()

#por planta

#Physalis peruviana
df_physalis = df_validacao[df_validacao["planta"] == "Physalis peruviana"]
plt.figure(figsize=(12, 6))
```

```
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_physalis,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Physalis peruviana", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_physalis.png", dpi=300)
plt.close()

#Solanum melongena
df_melongena = df_validacao[df_validacao["planta"] == "Solanum melongena"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_melongena,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Solanum melongena", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_melongena.png", dpi=300)
```

```
plt.close()

#Solanum tuberosum
df_tuberosum = df_validacao[df_validacao["planta"] == "Solanum tuberosum"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_tuberosum,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Solanum tuberosum", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_tuberosum.png", dpi=300)
plt.close()

#Arabidopsis thaliana
df_arabidopsis = df_validacao[df_validacao["planta"] == "Arabidopsis thaliana"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_arabidopsis,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Arabidopsis thaliana", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
```

```
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_arabidopsis.png", dpi=300)
plt.close()

#Heatmap
df_validacao["grupo"] = (
    df_validacao["planta"]
    + "_"
    + df_validacao["modelo"]
)

pivot = df_validacao.pivot_table(
    index="motivo",
    columns="grupo",
    values="contagem",
    fill_value=0
)


plt.figure(figsize=(14, 8))
sns.heatmap(
    pivot,
    annot=True,
    fmt="g",
    cmap="YlGnBu",
    linewidths=0.5,
    cbar_kws={'label': 'Contagem'} # Nomeia a barra de cores lateral
)

# plt.title("Distribuição dos Motivos Conservados por Planta e Modelo",␣
  ↪fontsize=14, fontweight="bold", pad=15)
plt.ylabel("Motivo", fontsize=12)
plt.xlabel("Grupo (Planta & Modelo)", fontsize=12)
plt.xticks(rotation=45, ha="right", fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/heatmap_distribuicao_dos_motivos_conservados.
  ↪png",dpi=300)
plt.close()
# plt.show()


print("========= Fim do Pipeline ============")
```